

Grounded in public OSS repo

michelangelo-ai/michelangelo

Uniflow SparkPlugin

A new Uniflow `TaskConfig` plugin letting custom Spark entrypoints — Scala/JVM jars or standalone PySpark scripts — run as workflow tasks without a Python wrapper function.

[Revision history](#) [What already exists](#) [Scope boundary](#) [Interface](#) [Starlark](#) [Local run](#) [Usage](#) [Alternatives](#) [RFC / PR](#)

Revision history

This design went through four passes. The most important correction: the original draft assumed new Cadence/backend infrastructure needed to be built. It didn't — the real engine already ships a submit+wait builtin pair backed by a schema that already supports both Scala/JVM and Python entrypoints.

1. **Original ask:** a `ScalaSpark` plugin for non-PySpark users, assuming new submission infrastructure needed to be built from scratch.
2. **Correction 1:** discovered the existing submit+wait Starlark builtin pair and job schema already support Scala/JVM. Redesigned as a thin wrapper over existing infra.
3. **Correction 2:** the "no Uniflow I/O" limitation isn't Scala-specific — it applies to any custom entrypoint. Renamed `ScalaSpark` → `SparkPlugin`, generalized to jar+class or `.py` file.

4. **Correction 3 (current):** re-grounded the entire design in the actual public `micangelo-ai/micangelo` repo — every path, class, and builtin referenced below is real, existing, public OSS code.

What already exists in the OSS repo

Verified by reading the source. `SparkPlugin` is not new infrastructure — it's a second, thinner `TaskConfig` implementation next to the existing `SparkTask` .

Piece	Path	Role
<code>TaskConfig</code> ABC	<code>python/micangelo/uniflow/core/task_config.py</code>	Base class every plugin implements: <code>get_binding()</code> , <code>get_config_binding()</code> , <code>pre_run()</code> , <code>post_run()</code> , <code>to_keywords()</code>
Existing wrapped Spark plugin	<code>plugins/spark/task.py</code> + <code>task.star</code>	<code>SparkTask</code> — the current pattern: a Python entrypoint runs as the Spark driver, so it can use Uniflow's typed I/O
Existing Ray plugin	<code>plugins/ray/task.py</code> + <code>task.star</code>	<code>RayTask</code> — second reference point for the same plugin shape
Submit/wait builtins	<code>go/worker/plugins/spark/starlark_module.go</code>	<code>spark.create_job(...)</code> / <code>spark.sensor_job(...)</code> — already used by <code>SparkTask</code> today
Job resource schema	<code>proto/api/v2/spark_job.proto</code>	<code>SparkJobSpec</code> already has <code>main_class</code> , <code>main_application_file</code> ("e.g. jar or

python file"), `deps.{jars,py_files}` —
already dual-language

Scope boundary

The design's central, deliberate trade-off — confirmed explicitly during review.

✅ SparkTask (existing, unchanged)

Python entrypoint runs as the actual Spark driver process, so it can read/write through Uniflow's I/O registry — full typed input/output between workflow steps, same as `RayTask` .

⚠️ SparkPlugin (new)

Any custom entrypoint — jar or standalone PySpark script — whose driver process isn't invoked as a Uniflow task function. Returns terminal job status only, never a marshaled return value.

If a downstream step needs data the job produced, that data must be written by the job itself to a location the caller already knows (a table, a blob/S3 path passed via `args / spark_conf`) — not read back through the I/O registry.

Interface — `SparkPlugin` dataclass

`python/michelangelo/uniflow/plugins/spark/plugin_task.py`

```
@dataclass
class SparkPlugin(TaskConfig):
    application_file: str
```

```

main_class: Optional[str] = None
args: List[str] = field(default_factory=list)
driver_cpu: Optional[int] = None
driver_memory: Optional[str] = None
executor_cpu: Optional[int] = None
executor_memory: Optional[str] = None
executor_instances: Optional[int] = None
spark_conf: Dict[str, str] = field(default_factory=dict)
deps_jars: List[str] = field(default_factory=list)
deps_py_files: List[str] = field(default_factory=list)
namespace: str = "default"

def get_binding(self) -> TaskBinding:
    return _binding

@classmethod
def get_config_binding(cls) -> TaskBinding:
    return _config_binding

def pre_run(self):
    """No-op -- the entrypoint manages its own runtime."""

def post_run(self):
    """No-op -- see pre_run()."""

```

`to_keywords()` is inherited unchanged from `TaskConfig` — no hand-written AST code needed, unlike an earlier draft of this design.

Field mapping onto `SparkJobSpec`

SparkPlugin field

SparkJobSpec field

<code>application_file</code>	<code>spec.main_application_file</code>
<code>main_class</code>	<code>spec.main_class</code> (optional)
<code>deps_jars</code> / <code>deps_py_files</code>	<code>spec.deps.jars</code> / <code>spec.deps.py_files</code>
<code>driver_cpu</code> / <code>driver_memory</code>	<code>spec.driver.pod.resource.{cpu,memory}</code>
<code>executor_cpu</code> / <code>executor_memory</code> / <code>executor_instances</code>	<code>spec.executor.pod.resource.{cpu,memory}</code> / <code>spec.executor.instances</code>

Starlark implementation sketch

`python/michelangelo/uniflow/plugins/spark/plugin_task.star` — mirrors `SparkTask` 's existing `task.star` : build a spec, `spark.create_job` it, `spark.sensor_job` it to a terminal state, minus the retry/cache logic that only makes sense for a wrapped Python entrypoint.

```
def spark_plugin_task(task_path, alias=None, application_file="", main_class=None, ...):
    def callable(*call_args, **call_kwargs):
        spec = { "driver": {...}, "executor": {...}, "mainApplicationFile": application_file, ... }
        if main_class:
            spec["mainClass"] = main_class
        job = {"kind": "SparkJob", "metadata": {...}, "spec": spec}

        response = spark.create_job(job)
        created = response["sparkJob"]
        final = spark.sensor_job(job=created, assert_condition_type=spark.succeeded_condition_type)

        # No result_url / io_read_json step -- job status only.
```

```
        report_progress(task_path=task_path, task_state=..., ...)
    return final
return callable
```

Local run (Python) implementation

`python/michelangelo/uniflow/plugins/spark/local.py` — shells to a local `spark-submit` using the same config object as the remote path, following Airflow's `SparkSubmitHook` precedent (always `spark-submit`, only `--master` differs).

```
def run_spark_submit_local(config: "SparkPlugin", extra_args: list = None) -> dict:
    cmd = ["spark-submit", "--master", "local[*]"]
    if config.main_class:
        cmd += ["--class", config.main_class]
    if config.deps_jars:
        cmd += ["--jars", ",".join(config.deps_jars)]
    if config.deps_py_files:
        cmd += ["--py-files", ",".join(config.deps_py_files)]
    cmd += [config.application_file, *config.args, *(extra_args or [])]
    result = subprocess.run(cmd, capture_output=True, text=True, check=True)
    return {"stdout": result.stdout, "returncode": result.returncode}
```

Usage example

```
# Scala/JVM
_scala_config = SparkPlugin(
```

```

    application_file="s3://my-bucket/my-app-1.0.jar",
    main_class="com.example.MySparkApp",
    args=["--date", "2026-07-02"],
    executor_cpu=4, executor_memory="4G", executor_instances=10,
)

@task(config=_scala_config)
def my_scala_spark_task(date: str) -> dict:
    return run_spark_submit_local(_scala_config, extra_args=["--date", date])

# Custom PySpark -- same plugin, no main_class, uses deps_py_files
_pyspark_config = SparkPlugin(
    application_file="s3://my-bucket/custom_job.py",
    deps_py_files=["s3://my-bucket/shared_utils.zip"],
)

@task(config=_pyspark_config)
def my_custom_pyspark_task(date: str) -> dict:
    return run_spark_submit_local(_pyspark_config, extra_args=["--date", date])

```

Alternatives rejected

- **Bypass the `spark` plugin, build against raw Kubernetes APIs** — would duplicate real production submit/wait logic that already exists and is already correct.
- **Add a `local_callable` method to the `TaskConfig` ABC** — breaks the ABC contract for `RayTask` / `SparkTask` with no benefit to them.
- **Keep it Scala-only (`ScalaSpark`), add a separate `PySparkPlugin` later** — the mechanism and limitation are identical regardless of language; splitting would duplicate the whole wrapper for no benefit.

RFC / PR

This design has been submitted as an RFC to the public `micangelo-ai/enhancements` repo:

github.com/micangelo-ai/enhancements/pull/15